

Practical 10

Inodes, hard links and symbolic links

```
chmod +x inode_links_demo.sh
./inode_links_demo.sh
```

Use `ls -li`, `stat`, and `find` to inspect inode numbers.

mmap file I/O

```
gcc -Wall -Wextra mmap_file.c -o mmap_file
./mmap_file
cat mmap_demo.txt
```

`MAP_SHARED` allows writes through the mapping to be reflected in the underlying file (after synchronization as appropriate). Memory-mapped I/O avoids an explicit read/write loop for the mapped region, but introduces mapping and page-management considerations.

Inode*links*demo.sh-

```
#!/bin/bash
```

```
set -e
```

```
rm -f original.txt hardlink.txt symlink.txt
```

```
printf "inode demonstration\n" > original.txt
```

```
echo "=== Original ==="
```

```
ls -li original.txt
```

```
stat original.txt
```

```
ln original.txt hardlink.txt
```

```
ln -s original.txt symlink.txt
```

```
echo
```

```
echo "=== After creating hard and symbolic links ==="
```

```
ls -li original.txt hardlink.txt symlink.txt
```

```
echo
```

```
echo "=== find by inode of original ==="
```

```
INODE=$(stat -c '%i' original.txt)
```

```
find . -maxdepth 1 -inum "$INODE" -printf '%f\n'
```

```
echo
```

```
echo "Hard link shares the same inode as the original."
```

```
echo "Symbolic link has its own inode and stores a pathname."
```

```
Mmap_file.c-
```

```
#define _GNU_SOURCE
```

```
#include <fcntl.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <sys/mman.h>
```

```
#include <sys/stat.h>
```

```
#include <unistd.h>
```

```
int main(void) {
```

```
    const char *name = "mmap_demo.txt";
```

```
    const char *initial = "Hello from mmap!\n";
```

```
    size_t len = strlen(initial);
```

```

int fd = open(name, O_RDWR | O_CREAT | O_TRUNC, 0644);

if (fd < 0) { perror("open"); return 1; }

if (ftruncate(fd, (off_t)len) < 0) { perror("ftruncate"); close(fd); return 1; }

if (write(fd, initial, len) != (ssize_t)len) { perror("write"); close(fd); return 1; }

char *p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

if (p == MAP_FAILED) { perror("mmap"); close(fd); return 1; }

printf("Before: %.*s", (int)len, p);

const char replacement[] = "MMAP changed!\n";

memcpy(p, replacement, len);

if (msync(p, len, MS_SYNC) < 0) perror("msync");

printf("After: %.*s", (int)len, p);

munmap(p, len);

close(fd);

puts("Read the file after unmapping:");

int r = open(name, O_RDONLY);

if (r >= 0) {

    char buf[128] = {0};

    ssize_t n = read(r, buf, sizeof(buf)-1);

    if (n > 0) write(STDOUT_FILENO, buf, (size_t)n);

    close(r);

}

return 0;

}

```